Portfolio — Technical Handoff

Repository: `git@github.com:Marnie0/Portfolio.git` **Production:** https://portfolio-ih18.vercel.app **Host:** Vercel (auto-deploy from `main`) **Backend:** Supabase (Postgres + Auth + Storage), project ref in `.env.local`

Written for a developer who has never seen this codebase. Names are exact.

---

1. Framework & language stack

Frontend framework

React 19.1 via Next.js 15.5.25 — not standalone React, not Vite. The **App Router** (`app/`), not the Pages Router. There is no `pages/` directory.

React Server Components are the default. A component is only client-side if its file starts with `'use client'`.

Every client component in the codebase (18 files — this is the complete list):

```
components/admin/AchievementForm.tsx     components/admin/SkillGroupForm.tsx
components/admin/ArticleForm.tsx         components/admin/SmallForms.tsx
components/admin/DeleteButton.tsx        components/layout/Navbar.tsx
components/admin/EducationForm.tsx       components/layout/ThemeProvider.tsx
components/admin/LanguageForm.tsx        components/layout/ThemeToggle.tsx
components/admin/LoginForm.tsx           components/sections/ContactForm.tsx
components/admin/ProjectForm.tsx         components/ui/MotionProvider.tsx
components/admin/ServiceForm.tsx         components/ui/Reveal.tsx
components/admin/SiteSettingsForm.tsx    components/ui/SkillCard.tsx
```

Everything else — including every `components/sections/*.tsx` except `ContactForm.tsx` — is a Server Component and queries Supabase directly.

Rendering strategy per route

Route	Strategy	Directive
`/`	**ISR**, 60s	`export const revalidate = 60` in `app/page.tsx:16`
`/articles`	**ISR**, 60s	`app/articles/page.tsx:13`
`/articles/[slug]`	**SSG + ISR**, 60s	`generateStaticParams()` prerenders published slugs; `revalidate = 60`

Route	Strategy	Directive
`/admin/login`	Static	no directive
`/admin/**` (protected)	**Dynamic (SSR)**	`export const dynamic = 'force-dynamic'` on the layout and every page
`/api/contact`	Dynamic route handler	POST only
`/sitemap.xml`, `/robots.txt`, `/manifest.webmanifest`	Static, generated from `app/sitemap.ts`, `app/robots.ts`, `app/manifest.ts`	

The admin is `force-dynamic` because it is per-session by definition; caching it would leak one request's data into another's.

Language / TypeScript

TypeScript 5.9.3. No JavaScript source files. `tsconfig.json`:

```
{
  "target": "ES2022",
  "lib": ["dom", "dom.iterable", "esnext"],
  "strict": true,            // strict mode IS on
  "noEmit": true,
  "module": "esnext",
  "moduleResolution": "bundler",
  "jsx": "preserve",
  "isolatedModules": true,
  "incremental": true,
  "skipLibCheck": true,
  "allowJs": true,
  "resolveJsonModule": true,
  "esModuleInterop": true,
  "plugins": [{ "name": "next" }],
  "paths": { "@/*": ["./*"] }   // @/lib/... resolves from the repo root
}
```

`npx tsc --noEmit` is the type check. It passes at every commit.

Styling

Tailwind CSS 3.4.19 only. No CSS Modules, no styled-components. One global stylesheet: **`app/globals.css`** (252 lines), which holds the design tokens and the long-form article styles.

`tailwind.config.ts` customisations:

- `darkMode: 'class'` — driven by `next-themes`
- **Colours** are CSS variables holding *space-separated RGB channels*, wrapped by `const token = (name) => \` rgb(var(--${name}) /)` so every token supports opacity modifiers ( bg-surface/60 ). Tokens: bg , surface , surface-muted , fg , muted , border , accent , accent-fg , accent-text , accent-soft`.
- **Fluid display sizes**: `display-sm` and `display` use `clamp()`.
- `maxWidth.content = 72rem` , `maxWidth.prose = 65ch`
- `borderRadius.4xl = 2rem`
- `transitionTimingFunction.out-expo = cubic-bezier(0.16, 1, 0.3, 1)`
- Keyframes `rise` , `slide` , `nudge` — `slide` animates transform only, never opacity, because the hero `<h1>` is the LCP element and Chrome will not count an `opacity: 0` element as painted.

Values live in `app/globals.css` under `:root` (light) and `.dark`.

Animation

Framer Motion 12.23.12, imported in exactly four files:

File	Use
`components/ui/MotionProvider.tsx`	Wraps the app in `<MotionConfig reducedMotion="user">` and `LazyMotion`
`components/ui/Reveal.tsx`	The scroll-reveal primitive — `whileInView` with `viewport={{ once: true, margin: '0px 0px -80px 0px' }}`
`components/ui/SkillCard.tsx`	Staggered chip entrance via variants
`components/layout/Navbar.tsx`	Scroll-progress bar (`useScroll` + `useSpring`) and the active-link underline

There are **no page transitions**. Hover states are CSS, not Framer.

`Reveal` uses the `m` import (not `motion`) plus `LazyMotion` , which is why only the `domAnimation` feature set is loaded. **Consequence:** layout animations (`layoutId`) are unavailable — `Navbar.tsx` has a comment explaining the underline is per-link for this reason.

Backend / data layer

Supabase via `@supabase/supabase-js 2.115.0` and `@supabase/ssr 0.12.6` . **Three distinct clients**, deliberately separated:

File	Client	Used by	Why
`lib/supabase/public.ts`	`createClient()` , `persistSession: false`	All public page reads	**Cookie-free**, so pages stay statically renderable / ISR-cacheable

File	Client	Used by	Why
`lib/supabase/server.ts`	`createServerClient()` from `@supabase/ssr`, wired to `next/headers` `cookies()`	Admin pages + all Server Actions	Carries the session, so RLS sees `authenticated`
`lib/supabase/browser.ts`	`createBrowserClient()`	`LoginForm`, image uploads in `ArticleForm` / `ProjectForm`	Login and direct-to-Storage uploads

`lib/supabase/config.ts` centralises env reading, trims values, and exports `isSupabaseConfigured`.

Both patterns are used: Server Components call the Supabase client directly for reads; all writes go through Server Actions. There are no API routes for data — `/api/contact` is the only route handler.

Email

Resend, called with plain `fetch` — the `resend` npm package is *not* a dependency. `app/api/contact/route.ts` POSTs to `https://api.resend.com/emails` with `Authorization: Bearer ${RESEND_API_KEY}` and `signal: AbortSignal.timeout(10_000)`. The route returns 200 **only** if Resend returns a message `id`; a 2xx with no id becomes a 502 so the form can never show a false success.

Tooling

- **Package manager: npm** (`package-lock.json` is committed; no pnpm/yarn lockfile)
- **Node: not pinned.** No `engines` field, no `.nvmrc`. Local dev is on v18.19.1; Vercel uses its own default. `@supabase/supabase-js` prints a deprecation warning on Node ≤20 — harmless.
- `sharp 0.33.5` is a direct dependency for image optimisation.
- **`npm run lint` is broken** — the script is `eslint .` but ESLint is not installed and there is no config. Next skips linting during build as a result.

---

2. Architecture overview

Folder layout

```
app/                          App Router
  layout.tsx                  Root layout. async — fetches settings for metadata,
                              JSON-LD, and Navbar props. generateMetadata() is async.
  page.tsx                    Home page. revalidate = 60. Composes the sections.
  globals.css                 Design tokens + .article-body + highlight.js theme
  robots.ts sitemap.ts manifest.ts icon.svg apple-icon.png
  articles/page.tsx           Public article list
  articles/[slug]/page.tsx    Public article, generateStaticParams + generateMetadata
  api/contact/route.ts        POST → Resend
  admin/
    actions.ts                'use server' — ARTICLE actions only
    content-actions.ts        'use server' — every other content action
    login/page.tsx            Public. The only login on the site.
    (protected)/              Route group — URL is /admin/..., the group adds auth
      layout.tsx              getUser() guard + admin chrome + section nav
      page.tsx                Articles dashboard (= /admin)
      new/, edit/[id]/        Article editor
      achievements/ education/ services/ skills/ projects/ site/

components/
  sections/                   One file per home-page section. All Server
                              Components except ContactForm.tsx
  admin/                      Admin UI. All client components except RowControls.tsx
  layout/                     Navbar, Footer, ThemeProvider, ThemeToggle
  ui/                         Icon, Section, Reveal, MotionProvider, SkillCard
  articles/Markdown.tsx       Shared (no 'use client') — see note below

lib/
  site.ts                     Compiled fallback for site config + navLinks + initials
  content.ts                  Compiled fallback for all list content
  articles.ts                 Article queries + slugify + formatArticleDate + readingTime
  content-db/                 One module per section — the DB read layer
  supabase/                   config, public, server, browser clients

supabase/                     Every migration, numbered, in run order
middleware.ts                 Session refresh + /admin gate. matcher: ['/admin/:path*']
HANDOFF.md / HANDOFF.pdf      Non-technical owner's guide
```

`components/articles/Markdown.tsx` deliberately has no `'use client'`. It is a *shared* component: the public article page renders it on the server with zero client JS, while `ArticleForm` (a client component)

bundles the same code for its live preview. Adding `'use client'` would ship react-markdown to every article reader for no reason.

The `lib/content-db/` fallback pattern

Six modules — `achievements.ts`, `education.ts`, `services.ts`, `skills.ts`, `projects.ts`, `settings.ts`. Every one follows the same shape:

```
function fallbackRows(): T[] { /* maps lib/content.ts into row shape */ }

export async function getX(): Promise<T[]> {
  if (!publicSupabase) return fallbackRows();          // not configured

  const { data, error } = await publicSupabase
    .from('table')
    .select(COLUMNS)
    .eq('visible', true)
    .order('sort_order', { ascending: true });

  if (error) {                                          // unreachable / query failed
    console.error('[x] falling back to compiled content:', error.message);
    return fallbackRows();
  }
  return (data ?? []) as T[];                           // zero rows → render empty
}
```

The critical distinction: the fallback fires on **error or missing config only** — never on an empty result set. Deleting every project really empties the section; it does not resurrect the compiled copy. This is not try/catch — `supabase-js` returns `{ data, error }` rather than throwing.

Why it exists: before the migration, content was compiled in and the site could not fail to render. Moving to a database introduced the possibility of a blank portfolio during an outage. Verified by pointing `NEXT_PUBLIC_SUPABASE_URL` at an unreachable host: the home page still returned 200 with every section populated, and all ten data sources logged their fallback.

`getSiteSettings()` in `lib/content-db/settings.ts` goes further — it **merges field by field** over `fallbackSettings()`, so a single blank column falls back rather than blanking the hero.

Each module also exports its column list as a constant (`ACHIEVEMENT_COLUMNS`, `EDUCATION_COLUMNS`, `SERVICE_COLUMNS`, `SKILL_GROUP_COLUMNS`, `LANGUAGE_COLUMNS`, `PROJECT_COLUMNS`) so the public read and the admin read cannot drift apart.

End-to-end data flow

Public read:

```
request → app/page.tsx (RSC, ISR 60s)
        → <Projects/> (async RSC)
        → getProjects()            lib/content-db/projects.ts
        → publicSupabase           cookie-free anon client
        → PostgREST + RLS "public reads visible"
        → ok?  data → render
          err? console.error → fallbackRows() from lib/content.ts → render
```

Admin write:

```
<form action={saveProject}>        components/admin/ProjectForm.tsx ('use client')
  → POST with Next-Action header
  → saveProject()                  app/admin/content-actions.ts ('use server')
  → requireAdmin() → createServerSupabase().auth.getUser()
      no user → redirect('/admin/login')
  → parse FormData, validate
  → supabase.from('projects').update|insert   RLS sees `authenticated`
  → revalidatePath('/') + revalidatePath('/admin/projects')
  → redirect('/admin/projects')
```

---

3. Database schema

Eleven tables in `public`. All migrations are committed in **`supabase/`** and must run in numeric order:

```
00_articles.sql     articles + set_updated_at() + article-images bucket
01_schema.sql       education, skill_groups, spoken_languages, services,
                    projects, achievements
02_policies.sql     RLS + triggers + indexes for those six, + content-images bucket
03_seed.sql         23 seed rows
04_site_schema.sql  site_settings, about_facts, hero_stats, social_links + RLS
05_site_seed.sql    settings row + 4 facts + 2 stats + 2 socials
06_telegram.sql     ALTER TABLE site_settings ADD telegram_url
```

`00_articles.sql` **must run first** — it defines `public.set_updated_at()`, which every later trigger reuses.

Shared column conventions

All list tables carry: `id uuid primary key default gen_random_uuid()`, `visible boolean not null default true`, `sort_order integer not null default 0`, `created_at timestamptz not null default now()`, `updated_at timestamptz not null default now()`, a `<table>_set_updated_at` BEFORE UPDATE trigger, and an index `<table>_order_idx on (visible, sort_order)`.

`articles`

Column	Type	Null	Default
`id`	uuid	no	`gen_random_uuid()` — PK
`title`	text	no	
`slug`	text	no	**UNIQUE**
`excerpt`	text	yes	
`content`	text	no	`''`
`cover_image_url`	text	yes	
`published`	boolean	no	`false`
`published_at`	timestamptz	yes	
`created_at`	timestamptz	no	`now()`
`updated_at`	timestamptz	no	`now()`

Index: `articles_published_idx on (published, published_at desc)`. **No `visible` or `sort_order`** — articles order by `published_at desc`, then `created_at desc`. Verified by probing the live table.

`education`

`degree` text NOT NULL · `institution` text NOT NULL · `period` text NOT NULL · `location` text **nullable** · `description` text NOT NULL `''` · `highlights` text[] NOT NULL `'{}'` + shared columns.

`location` is intentionally nullable: `Education.tsx` omits the line entirely when null, and an empty string would render a blank row.

`skill_groups`

`title` text NOT NULL · `skills` text[] NOT NULL `'{}'` + shared columns.

`spoken_languages`

`name` text NOT NULL · `level` text NOT NULL + shared columns.

`services`

`title` text NOT NULL · `description` text NOT NULL `''` · `deliverables` text[] NOT NULL `'{}'` · `icon` text NOT NULL `'code'` + shared columns.

`icon` is a free-text column validated **in application code**, not by a CHECK constraint — `serviceIcon()` in `lib/content-db/services.ts` falls back to `'code'` for unknown values, and the admin uses a `<select>`.

`projects`

`title` text NOT NULL · `category` text NOT NULL `''` · `year` text NOT NULL `''` · `summary` text NOT NULL `''` · `focus` text NOT NULL `''` · `tech` text[] NOT NULL `'{}'` · `image_url` text **nullable** · `image_alt` text NOT NULL `''` · `live_url` text **nullable** · `github_url` text **nullable** · `featured` boolean NOT NULL `false` + shared columns.

`year` is text, not integer — the design shows values like `"Ongoing"` elsewhere and consistency was preferred over type precision.

`achievements`

`title` text NOT NULL · `issuer` text NOT NULL `''` · `year` text NOT NULL `''` · `description` text NOT NULL `''` · `type` text NOT NULL `''` + shared columns.

`site_settings` — singleton

```
id integer primary key default 1,
constraint site_settings_singleton check (id = 1)
```

That CHECK makes a second row **physically impossible**; no application code enforces it. 23 columns total:

`name` · `short_name` · `role` · `tagline` · `description` · `location` · `availability` · `email` · `phone_display` · `phone_tel` · `whatsapp_url` · `telegram_url` · `resume_url` (default `'/CV.pdf'`) · `hero_cta_primary` (default `'View work'`) · `hero_cta_secondary` (default `'Contact me'`) · `hero_resume_label` · `about_eyebrow` (default `'About'`) · `about_lead` · `about_paragraphs` **text[]** · `contact_eyebrow` (default `'Contact'`) · `contact_title` · `contact_description` · `updated_at`.

All text columns are `NOT NULL DEFAULT ''`. **No `visible`, no `sort_order`** — this table is the page itself, not a list.

`about_facts`

`label` text NOT NULL · `entries` **text[]** NOT NULL `'{}'` + shared columns.

`entries` reproduces the original design exactly: length 1 renders inline, length > 1 renders stacked lines (`About.tsx`). No extra flag needed.

`hero_stats`

`value` text NOT NULL · `label` text NOT NULL + shared columns.

`social_links`

`label` text NOT NULL · `url` text NOT NULL · `icon` text NOT NULL `'github'` · `display` text NOT NULL `''` + shared columns.

Feeds **four** places at once: hero icon row, footer icon row, the Contact channel list, and the JSON-LD `sameAs` array in `app/layout.tsx`.

Foreign keys

There are none. Every table is independent. `skill_groups.skills` is a `text[]` rather than a child table — the UI renders a flat chip list, so an array matched the design and avoided six extra admin screens.

RLS policies

RLS is enabled on all eleven tables. Ten of them use an identical four-or-five policy set (from `02_policies.sql` / `04_site_schema.sql`):

Policy name	Command	Roles	Condition
`public reads visible`	SELECT	`anon, authenticated`	`using (visible = true)`
`admin reads all`	SELECT	`authenticated`	`using (true)`
`admin inserts`	INSERT	`authenticated`	`with check (true)`
`admin updates`	UPDATE	`authenticated`	`using (true) with check (true)`
`admin deletes`	DELETE	`authenticated`	`using (true)`

Multiple SELECT policies are **OR-ed**, so a signed-in user sees hidden rows too.

`articles` uses the same shape with `using (published = true)` instead of `visible`, and its policies are named `public reads published articles`, `admin reads all articles`, `admin inserts articles`, etc.

`site_settings` differs — no `visible` column, and **no DELETE policy** (the singleton must not be deletable):

| `public reads settings` | SELECT | `anon, authenticated` | `using (true)` | | `admin inserts settings` | INSERT | `authenticated` | `with check (true)` | | `admin updates settings` | UPDATE | `authenticated` | `using (true) with check (true)` |

Storage (`storage.objects`), two buckets, both `public = true`: `article-images` and `content-images`. Each has `public reads …` (SELECT, anon+authenticated), `admin uploads …` (INSERT, authenticated), `admin deletes …` (DELETE, authenticated), all keyed on `bucket_id`.

The security model in one sentence: `authenticated` == the owner, because public sign-ups are disabled in Supabase Auth. Verified live — `POST /auth/v1/signup` returns `{"code":422,"error_code":"signup_disabled"}`. **If sign-ups are ever re-enabled, every one of these policies becomes wrong**, since any account could then write.

Known schema issues

1. **`hero_resume_label` default drift.** `supabase/04_site_schema.sql` says `default 'Résumé'`, but the SQL actually pasted into the editor used `default 'Resume'` (unaccented — the accent was stripped

while debugging a paste-corruption issue). **The live row value is `'Résumé'` and correct**; only the column *default* may differ, and it applies only to a row inserted without that column — impossible here, since the table is a singleton that already exists. Cosmetic; committed file is the intended state.

2. **`articles` DDL was missing from the repo** until now. Reconstructed as `supabase/00_articles.sql`, verified column-by-column against the live table and parsed against the real Postgres grammar.
3. **No CHECK constraints on `icon` columns.** Validation is application-side only (`serviceIcon()`, `socialIcon()`). Writing an arbitrary icon string via SQL renders a fallback glyph rather than failing.

---

4. Server Actions & API surface

Two `'use server'` files. **Nothing else in the codebase is a Server Action.**

`app/admin/actions.ts` — articles

Function	Signature	Mutates	Invoked from
`saveArticle`	`(prev: ActionState, fd: FormData) => Promise<ActionState>`	`articles` insert/update	`useActionState` in `ArticleForm.tsx`
`deleteArticle`	`(fd: FormData) => Promise<void>`	`articles` delete	`<form action={deleteArticle}>` in `app/admin/(protected)/page.tsx`
`togglePublished`	`(fd: FormData) => Promise<void>`	`published`, `published_at`	`<form action={togglePublished}>`
`signOut`	`() => Promise<void>`	auth session	`<form action={signOut}>` in `(protected)/layout.tsx`

`saveArticle` reads `intent` from the submitter button (`draft` / `publish`) and preserves the original `published_at` when re-publishing, so editing a live post does not reorder the list.

`app/admin/content-actions.ts` — everything else

Generic, table-driven:

Function	Signature	Notes
`deleteRow`	`(fd: FormData) => Promise<void>`	reads `table`, `id`
`toggleVisible`	`(fd: FormData) => Promise<void>`	reads `table`, `id`, `next`
`moveRow`	`(fd: FormData) => Promise<void>`	reads `table`, `id`, `direction` (`up` / `down`)

All three take the **table name from a form field**, so they validate it through `assertTable()` against `EDITABLE_TABLES`:

```
const EDITABLE_TABLES = ['achievements','education','services','skill_groups',
  'spoken_languages','projects','about_facts','hero_stats','social_links'] as const;
```

An unknown table throws. RLS would refuse an anonymous caller anyway; this is defence in depth.

`moveRow` swaps `sort_order` with the neighbour rather than renumbering — two writes, no gaps. It handles neighbours sharing a value:

```
const aOrder = a.sort_order === b.sort_order ? b.sort_order + direction : b.sort_order;
```

Verified against real rows: 9/9 cases pass including both end-stops and the tie.

Per-section save actions, all `(prev: ActionState, fd: FormData) => Promise<ActionState>`:

`saveAchievement` · `saveEducation` · `saveService` · `saveSkillGroup` · `saveLanguage` · `saveProject` · `saveSiteSettings` · `saveAboutFact` · `saveHeroStat` · `saveSocialLink`

Shared internals (not exported — a `'use server'` file may only export async functions):

- `requireAdmin()` — `getUser()`, `redirect('/admin/login')` if absent. Uses `getUser()` not `getSession()`: the latter only reads the cookie and would trust a forged one.
- `parseLines(value)` — textarea → `text[]`, one item per line, blanks dropped.
- `nextSortOrder(supabase, table)` — `max(sort_order) + 1`.
- `revalidateContent(table)` — `revalidatePath('/')` + `revalidatePath('/admin/'+table)`.
- `missingColumnFrom(message)` — see below.

`saveSiteSettings` writes via `upsert({ id: 1, ... }, { onConflict: 'id' })` and **retries once without an unknown column** if the write fails, so a pending migration cannot block every other field. It parses the column name from both error shapes (PostgREST `PGRST204` schema-cache wording and Postgres `42703`), covered by 8 unit-tested cases.

`ActionState`

```
export type ActionState = { error: string } | null;
```

Success paths `redirect(...)`; failures return `{ error }`, rendered by the form. `redirect()` throws internally, so it must never be inside a `try`.

The "Server Action was not found on the server" error

Symptom: clicking ↑/↓ in `/admin/projects` threw `UnrecognizedActionError: Server Action "40c98d68862422d6cc2237f26dcd120c0be3f10e88" was not found`.

Root cause: *not a code bug.* Next.js gives each Server Action a **content-hashed ID** that changes on every build. A browser tab rendered by an earlier build posts an ID the current server no longer knows.

Diagnosis: the current build registered 17 IDs in `.next/server/server-reference-manifest.json`; the errored ID was not among them. Posting it directly returned **HTTP 404**, while a current ID returned **HTTP 200**.

Fix: hard-refresh the tab. No code change. It recurs in production for anyone holding `/admin` open across a deploy; refreshing clears it. Inherent to how Next versions Server Actions.

`/api/contact` — the only route handler

`POST` JSON `{ name, email, message, company }`. `company` is a **honeypot** — if filled, returns `200 {ok:true}` without sending, so bots get no signal.

Validation: name required, `EMAIL_PATTERN` regex, message ≥ 10 chars, and max lengths 100 / 200 / 5000 → `422 { errors }`. Missing `RESEND_API_KEY` → `500`. Resend rejection or a 2xx without an `id` → `502`. Message body is HTML-escaped by `escapeHtml()` before being embedded in the email.

Recipient resolution order: `CONTACT_TO_EMAIL` env → `site_settings.email` → `siteConfig.email`.

---

5. Admin panel implementation

Auth — four independent layers

1. **`middleware.ts`** — `matcher: ['/admin/:path*']`, so the public site pays no middleware cost. Refreshes the Supabase session cookie, then redirects to `/admin/login` when `getUser()` returns none, and away from `/admin/login` when it returns one. If `isSupabaseConfigured` is false it lets the request through, so the login page can explain the misconfiguration instead of redirect-looping.
2. **`app/admin/(protected)/layout.tsx`** — repeats `getUser()` server-side. Survives a middleware misconfiguration.
3. **Every mutating action** calls `requireAdmin()`. This is the layer that matters most: **a Server Action is its own POST endpoint and does not inherit middleware protection.**
4. **RLS** refuses anonymous writes at the database.

The `(protected)` **route group** is what allows `/admin/login` to sit outside the guard while `/admin`, `/admin/projects` etc. sit inside — parentheses do not appear in the URL.

Component breakdown

Component	Client?	Wiring
`components/admin/RowControls.tsx`	**Server**	Renders five sibling `<form>`s — two `moveRow` (up/down), one `toggleVisible`, a `<Link>` to edit, one `deleteRow`. Each passes `table` / `id` as `<input type="hidden">`. Props: `table, id, title, visible, editHref, isFirst, isLast`.

Component	Client?	Wiring
`components/admin/DeleteButton.tsx`	Client	Submit button with `window.confirm()`; calls `event.preventDefault()` on cancel. Client only because it needs `onClick`.
`components/admin/LoginForm.tsx`	Client	`signInWithPassword()` via browser client, then `router.replace('/admin')` + `router.refresh()`. Error message is deliberately generic so it cannot enumerate accounts.
`components/admin/ArticleForm.tsx`	Client	`useActionState(saveArticle)`. Markdown editor: split-pane at `lg`, tabbed below. Uploads to `article-images`. Inserts image Markdown **at the caret** via `textarea.selectionStart`.
`components/admin/ProjectForm.tsx`	Client	`useActionState(saveProject)` + upload to `content-images` under `projects/`.
`components/admin/SiteSettingsForm.tsx`	Client	`useActionState(saveSiteSettings)`. Grouped `<fieldset>`s: Identity / Contact details / Hero buttons / About / Contact section.
`components/admin/SmallForms.tsx`	Client	Three forms in one file — `AboutFactForm`, `HeroStatForm`, `SocialLinkForm`.
`AchievementForm` `EducationForm` `ServiceForm` `SkillGroupForm` `LanguageForm`	Client	Same `useActionState` shape.

`RowControls` being a **Server Component** is deliberate — passing a Server Action straight to `<form action={...}>` needs no client boundary.

Per-section status

Section	Route	Create	Edit	Delete	Reorder	Show/Hide
Articles	`/admin`	✅	✅	✅	n/a (date order)	✅ draft/publish
Achievements	`/admin/achievements`	✅	✅	✅	✅	✅
Education	`/admin/education`	✅	✅	✅	✅	✅
Services	`/admin/services`	✅	✅	✅	✅	✅
Skill groups	`/admin/skills`	✅	✅	✅	✅	✅
Spoken languages	`/admin/skills`	✅	✅	✅	✅	✅
Projects	`/admin/projects`	✅	✅	✅	✅	✅
Site settings	`/admin/site`	singleton	✅	n/a	n/a	n/a

Section	Route	Create	Edit	Delete	Reorder	Show/Hide
About facts	`/admin/site`	✅	✅	✅	✅	✅
Hero stats	`/admin/site`	✅	✅	✅	✅	✅
Social links	`/admin/site`	✅	✅	✅	✅	✅

Nothing is pending. Every section is fully database-backed.

---

6. Image handling

Buckets

Bucket	Public	Used by	Path convention
`article-images`	yes	`ArticleForm` — covers + inline	`${crypto.randomUUID()}-${safeName}` at the root
`content-images`	yes	`ProjectForm`	`projects/${crypto.randomUUID()}-${safeName}`

`safeName = file.name.replace(/[^a-zA-Z0-9.-]/g, '-').toLowerCase()`. Uploads use `{ cacheControl: '31536000', upsert: false }`. Public URL comes from `supabase.storage.from(bucket).getPublicUrl(path)`.

Deleting a row does not delete its uploaded file. Orphans accumulate in Storage and remain publicly reachable by direct URL. There is no cleanup job.

`next.config.mjs`

```
const supabaseImagePattern = {
  protocol: 'https',
  hostname: '*.supabase.co',
  pathname: '/storage/v1/object/public/**',
};
// images: { formats: ['image/avif','image/webp'], remotePatterns: [supabaseImagePattern] }
```

The bug that forced this shape: the original config derived the hostname from `process.env.NEXT_PUBLIC_SUPABASE_URL`. **Next evaluates `next.config.mjs` before loading `.env.local`**, so that variable was `undefined` and `remotePatterns` evaluated to `[]`. Proven by importing the config with and without the env loaded: `[]` vs the expected pattern. Every uploaded image then failed with *"hostname … is not configured under images"*, while the seeded `/public` images kept working because local paths need no allow-list — which is why it stayed hidden until the first real upload.

The wildcard removes the env dependency entirely. One pattern covers both buckets. Verified by fetching all 46 optimized images the home page requests — every one returned `200 image/jpeg`.

`ProjectImage` — `components/sections/Projects.tsx`

```
<div className="relative aspect-[16/10] w-full shrink-0 overflow-hidden rounded-2xl border borde
  {project.image_url && (
    <Image src={project.image_url} alt={project.image_alt} fill sizes={sizes}
           className="object-cover transition-transform duration-700 ease-out-expo group-hover:sc
  )}
</div>
```

Ratio pinned on the container, `fill` on the image. When `image_url` is null the box stays and renders empty, so a project without a cover still lines up.

`app/articles/page.tsx` uses the same approach at `aspect-[16/9]`.

---

7. Styling & layout

The project-card height bug

Symptom: cards in the Projects grid had mismatched heights; a card with a portrait cover left a large dead gap above its neighbour's buttons.

Root cause: the image container had **no aspect ratio of its own**. The `<Image>` declared `width={1600} height={1000}` with `className="h-full w-full object-cover"`, so once real dimensions loaded the block took the *file's* ratio. Measured in headless Chrome at 1440px:

```
test      image 662x827 (portrait) → media 612px → card 990
facebook  image 662x414            → media 307px → card 990
the town  image 662x414            → media 307px → card 754
```

Grid stretch forced row 1 to 990px, and that row sat 236px taller than row 2. The no-image branch used `aspect-[16/10]` — a third, inconsistent contract.

Fix: pin `aspect-[16/10]` on the container, switch the image to `fill`, and add `h-full` to the `<Reveal as="li">` grid item (without a definite height on the item, `h-full` on the `<article>` had nothing to resolve against).

After: all three cards 753px, all media blocks 306px, buttons on one line. Confirmed at 390px too — every media block measures ratio 1.60.

Note for future measurement work: the first measurement pass wrongly reported equal heights because images had not loaded, so the browser was still using the declared `1600x1000` ratio. Any layout

measurement must wait for `document.images` to complete.

Shared layout primitives

- **`components/ui/Section.tsx`** — `Section` (landmark `<section>`, `aria-labelledby={id + '-heading'}`, `max-w-content` wrapper, `py-20 sm:py-28 lg:py-32` unless `bare`) and `SectionHeading` (eyebrow + `<h2 id={id + '-heading'}>` + optional description).
- **`components/ui/Reveal.tsx`** — the only scroll animation. Props `delay`, `y`, `className`, `as` (`div|li|article|section|span`). Respects `useReducedMotion()`.
- **Card height consistency** comes from grid stretch + `h-full` on the `<article>`, plus `mt-auto` on the footer row to pin actions to the bottom.
- **`.article-body`** in `globals.css` styles rendered Markdown as one class rather than per-element overrides in the renderer.
- `<noscript>` in `app/layout.tsx` forces `[style*="opacity:0"]` elements visible, so the site is readable with JavaScript disabled.

---

8. Environment variables

Name	Public?	Purpose	Source
`NEXT_PUBLIC_SUPABASE_URL`	**Public** (browser)	Supabase project URL	Supabase → Project Settings → API → Project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	**Public** (browser)	Anon/publishable key	Same page → anon / public
`NEXT_PUBLIC_SITE_URL`	**Public**, build-time	`metadataBase`, sitemap, robots, canonical	Your own domain
`RESEND_API_KEY`	**Server only**	Contact-form delivery	resend.com → API Keys
`CONTACT_TO_EMAIL`	Server only, optional	Overrides recipient	Any address
`CONTACT_FROM_EMAIL`	Server only, optional	Sender; default `Portfolio <onboarding@resend.dev>`	Needs a Resend-verified domain

`NEXT_PUBLIC_*` are inlined at build time. Changing one in Vercel does nothing until a redeploy — this caused a real production build failure (see §12).

The `service_role` key is **not used anywhere** and must never be added. Admin writes go through the user's session, so RLS — not key secrecy — is the control.

---

9. Local dev setup

```
git clone git@github.com:Marnie0/Portfolio.git
cd Portfolio
npm install
cp .env.example .env.local     # fill in the values from §8
npm run dev                    # http://localhost:3000
```

Database, from scratch, in this order — `00` first, it defines `set_updated_at()`:

```
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/00_articles.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/01_schema.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/02_policies.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/03_seed.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/04_site_schema.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/05_site_seed.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase/06_telegram.sql
```

Connection string: Supabase → Project Settings → Database → Connection string → URI. Or paste each file into the SQL Editor — but see §12 on paste corruption.

Then create the admin user: Supabase → Authentication → Users → Add user, tick **Auto Confirm User**. Then **Authentication → Sign In / Providers → Email → turn OFF "Allow new users to sign up"**. Skipping that second step makes every RLS policy in §3 meaningless.

Seed data is idempotent — fixed UUIDs plus `on conflict (id) do nothing`.

> Do not run `npm run build` while `npm run dev` is running; they share `.next` and the dev server starts returning 500s.

---

10. Deployment & CI

- **Vercel**, GitHub integration, auto-deploys every push to `main`. No `vercel.json`; framework auto-detected. Build `npm run build`, install `npm install`.
- **No CI pipeline** — no GitHub Actions, no tests. The build is the only gate.
- Env vars are set in **Vercel → Settings → Environment Variables** (Config type for the `NEXT_PUBLIC_*` ones; they ship to the browser regardless). Preview and Production both point at the **same Supabase project** — a preview deploy writes to live data.
- **Reading status:** Vercel → Deployments, or the ✓/✗ on the GitHub commit. Failures: open the deployment → Build Logs → scroll to the **bottom**.
- **Rollback:** Deployments → an older successful one → ⋯ → **Promote to Production**.

- Deployment state is also queryable without the Vercel API: `GET https://api.github.com/repos/Marnie0/Portfolio/deployments` and `.../deployments/:id/statuses` return the state and the `environment_url`.

---

11. Git workflow

- **Single `main` branch.** No feature branches, no PRs, no tags.
- Commit messages: imperative subject, then a body explaining *why* and what was verified. Several record the exact measurements or error codes that proved a fix.
- Co-authorship trailers are present on commits made through Claude Code.

```
git status && git log --oneline -10
git add -A && git commit -m "…" && git push origin main   # triggers a deploy
git revert HEAD && git push origin main                   # safe undo
git reset --soft HEAD~1                                   # local, keeps changes
```

Reverting a migration is *not* covered by `git revert` — the SQL has already run. Write a compensating migration:

```
alter table public.site_settings drop column if exists telegram_url;
```

Never edit an applied migration file in place; the database will not match it.

---

12. Bugs hit during the build — root causes and fixes

1. Empty env var broke the production build

`TypeError: Invalid URL … input: ''` while collecting metadata for `/_not-found`. `process.env.X ?? fallback` only falls back on `null` / `undefined`; an env var set to an **empty string in Vercel** passed through and reached `new URL('')`. Locally the variable was *absent*, so the fallback worked — hence Vercel-only.

Fix: `resolveSiteUrl()` in `lib/site.ts` trims, treats blank as absent, adds a scheme when missing, wraps `new URL()` in try/catch, and returns `.origin`. Same class of bug fixed in `app/api/contact/route.ts` via `envOr()`.

2. Supabase SQL editor mangled large pastes

`DO $$ … $$` blocks failed with errors pointing at line numbers that did not exist in the file, and one paste arrived visibly truncated mid-statement.

Fix: `02_policies.sql` and `04_site_schema.sql` were rewritten as **plain static DDL** — no `DO` blocks, no `format()`, no dollar-quoting. `supabase/step04/` holds the same content split into five smaller files for pasting. **Recommendation: use `psql -f` instead of the web editor.**

3. `next/image` rejected Supabase uploads

See §6. Config evaluated before `.env.local` loads → empty `remotePatterns`.

4. Server Action not found

See §4. Stale client after a rebuild. No code change.

5. Project card heights

See §7. Unpinned image aspect ratio.

6. One missing column blocked all settings saves

Adding `telegram_url` to the form meant every save posted a column that did not exist until `06_telegram.sql` ran — so **no** setting could be saved.

Fix: `saveSiteSettings` retries once without the unknown column and reports which column was skipped, only when it actually held a value.

Remaining tech debt

Item	Impact
`npm run lint` is broken — `eslint .` with ESLint not installed	No linting. Build unaffected.
No tests of any kind	Verification is manual + type checking.
Node version not pinned	Local 18, Vercel newer.
Orphaned Storage files on delete	Grows unbounded; files stay public.
Preview deploys share the production database	A preview write is a real write.
`lib/content.ts` / `lib/site.ts` drift from the DB	By design, but the fallback shows increasingly stale content.
`hero_resume_label` default drift	Cosmetic; see §3.
No CHECK constraints on `icon` columns	Application-side validation only.
Contact route's 10s Resend timeout ≈ Vercel Hobby function limit	A slow Resend response may be cut off before the friendly error path runs. Consider 8s.
`supabase/step04/` duplicates `04_site_schema.sql`	Kept as a paste workaround; delete once `psql` is the norm.

13. Done vs. pending, file by file

Fully Supabase-backed

Section	Component	Data module	Table(s)
Hero	`components/sections/Hero.tsx`	`lib/content-db/settings.ts`	`site_settings`, `hero_stats`, `social_links`
About	`components/sections/About.tsx`	`settings.ts`	`site_settings`, `about_facts`
Education	`Education.tsx`	`education.ts`	`education`
Skills	`Skills.tsx`	`skills.ts`	`skill_groups`, `spoken_languages`
Services	`Services.tsx`	`services.ts`	`services`
Projects	`Projects.tsx`	`projects.ts`	`projects`
Achievements	`Achievements.tsx`	`achievements.ts`	`achievements`
Contact	`Contact.tsx`	`settings.ts`	`site_settings`, `social_links`
Footer	`components/layout/Footer.tsx`	`settings.ts`	`site_settings`, `social_links`
Navbar	`components/layout/Navbar.tsx`	via props from `app/layout.tsx`	`site_settings`
Metadata + JSON-LD	`app/layout.tsx`	`settings.ts`	`site_settings`, `social_links`
Articles	`app/articles/**`	`lib/articles.ts`	`articles`

Still hardcoded — and correctly so

File	Role
`lib/content.ts`	Outage fallback for all list sections
`lib/site.ts`	Outage fallback for site config; also owns `navLinks`, `initials`, `resolveSiteUrl()`
`components/sections/ArticlesCta.tsx`	Static copy. Not DB-backed.

File	Role
Section headings — "Where the foundations were laid", "The stack I reach for", "How I can help", "Projects I am proud of", "Milestones along the way"	Hardcoded in each section component. **About and Contact headings are DB-backed; these five are not.** An obvious next migration.
`app/sitemap.ts`, `app/robots.ts`	Use `siteConfig.url` (env-derived) — cannot be DB-backed, needed at build time
`lib/site.ts` → `navLinks`	Nav labels and order

Operational pending items

- `site_settings.telegram_url` is **empty** — the button is hidden until set.
- A project titled `test` may still exist as a hidden row; not publicly visible. Check `/admin/projects`.
- One orphaned image in `content-images/projects/` from that test.
- `articles` table is **empty** — the blog works but has no posts.

Things worth knowing that weren't asked

1. **`getUser()` vs `getSession()`.** Every auth check uses `getUser()`, which revalidates the JWT with Supabase. `getSession()` only decodes the cookie and would trust a forged one. Do not "optimise" this.
2. **The honeypot is silent.** `/api/contact` returns success for bot submissions rather than an error, so scrapers get no feedback.
3. **`rehype-raw` is deliberately NOT enabled** in `components/articles/Markdown.tsx`. Raw HTML inside an article is escaped, not executed.
4. **Nav links are root-relative** (`/#about`, not `#about`) so they work from `/articles`. On the home page the browser still treats them as same-document fragment navigation. `Navbar.tsx` derives scroll-spy ids with `.slice(2)`.
5. **`Reveal` server-renders at `opacity: 0`.** Without JS, content would be invisible — hence the `<noscript>` override in `app/layout.tsx`. Any new reveal-style animation needs the same consideration.
6. **`app/robots.ts` disallows `/admin`**, and `/admin/login` sets `robots: { index: false, follow: false }`.
7. **ISR means admin edits take up to 60s** to appear publicly, though the actions call `revalidatePath()` so it is usually immediate. A hard refresh of a cached page may still show stale content briefly.
8. **The image optimizer logs `LRUCache: calculateSize returned 0`** in dev. Known Next 15.5.x dev-only noise; images still serve.
9. **Fonts** are `Inter` and `Instrument_Serif` via `next/font/google`, self-hosted at build time, exposed as `--font-sans` / `--font-display`.
10. **`initialsFrom()` exists twice** — `lib/site.ts` (compiled name) and `lib/content-db/settings.ts` (DB name). Deliberate: the client-side Navbar receives the DB-derived value as a prop. Keep both in

sync if changed.